



Reentrancy Attack

Domain: **Blockchain & Smart-Contract Security**

A complete, beginner-friendly walk-through — every term explained

TEAM — GROUP 6

Rohit Patel	G25AIT2089	Pujan Chakraborty	G25AIT2076
Sharvan Vittala	G25AIT2099	Amit Singh	G25AIT2007
Sudeb Ghosh	G25AIT2113	Vishnu Priya	G25AIT2128
Kosuru Yuvaraj	G25AIT2054		

What This Deck Covers (Read Top to Bottom)

A

Foundations

Blockchain, Ethereum, ETH, smart contracts, gas — the vocabulary

B

Key Building Blocks

Functions, state, msg.sender, fallback, external calls

C

The Vulnerability

What reentrancy means, word by word

D

Attack — Step by Step

Every line of the recursive drain explained

E

Code Walkthrough

Vulnerable vs Fixed, line by line

F

Demo + Defenses + Roles

Foundry on your laptop, prevention, who does what

The Absolute Basics — Part 1

Blockchain = a shared digital ledger

A database copied across thousands of computers. Once data is written, it cannot be secretly changed. No single owner — everyone holds the same copy.

Ethereum = a programmable blockchain

Like Bitcoin, but it can also RUN programs, not just move money. Those programs are called smart contracts.

ETH = Ether

The currency of Ethereum (like rupees on a bank network). 1 ETH is divisible. The smallest unit is 'wei'. 1 ETH = 1,000,000,000,000,000,000 wei (10^{18}).

Wei = smallest unit of ETH

Like paise to a rupee, but tiny. Contracts count money in wei to avoid decimals. 1 ETH = 10^{18} wei.

Smart Contract = self-running program on Ethereum

Code that lives on the blockchain. It holds ETH and runs exactly as written — no human can stop it mid-way. 'Code is law.'

Wallet / Address = a 42-character account ID

Your identity on Ethereum, e.g. 0xA11CE... Both people AND contracts have addresses and can hold ETH.

The Absolute Basics — Part 2

EVM = Ethereum Virtual Machine

The 'engine' that runs every smart contract on every node. Think of it as a worldwide shared computer that executes the contract code.

Transaction (tx) = a signed instruction

A request you send to the blockchain, e.g. 'send 1 ETH' or 'call withdraw()'. It is atomic: it fully succeeds OR fully reverts (undoes).

Gas = fee + work meter

Every operation costs 'gas', paid in ETH. It stops infinite loops — run out of gas and the tx reverts. More work = more gas.

Revert = undo everything

If a tx fails a check or runs out of gas, the EVM rolls back ALL changes as if it never happened. This is our main defense later.

State = the contract's saved memory

Data a contract permanently stores between calls — e.g. how much ETH each user has. Stored in 'state variables'.

Deploy = publish a contract

Uploading your compiled contract onto Ethereum so it gets its own address and can be used by anyone.

Solidity Words You'll See in the Code

Solidity = the main programming language for writing Ethereum smart contracts (file ending: .sol).

function = a named block of code

An action the contract can perform, e.g. deposit() or withdraw(). Callers 'invoke' it to make something happen.

mapping = a key → value table

balances[address] => uint. Like a dictionary: 'for this address, store this number'. We track each user's balance here.

msg.sender = who called this function

The address making the current call. If Eve calls withdraw(), msg.sender = Eve's address. Crucial for the attack.

msg.value = ETH sent with the call

How much ETH (in wei) was attached to this call. deposit() reads msg.value to credit your balance.

require(x) = a guard / check

If condition x is false, REVERT immediately. e.g. require(bal > 0) stops withdrawing when you have nothing.

uint = unsigned integer

A whole number that cannot be negative (0,1,2,...). Used for balances and amounts. 'uint256' = 256-bit number.

The Two Words That Make the Attack Possible

1. External Call

```
msg.sender.call{value: bal}("")
```

This is how a contract SENDS ETH to another address.

The danger: when you send ETH to an address that is itself a CONTRACT, that contract's code automatically runs.

So 'sending money' can secretly 'hand control' to the receiver — who may be malicious.

2. Fallback / receive()

```
receive() external payable { ... }
```

A special function that runs AUTOMATICALLY whenever a contract receives ETH.

You don't call it on purpose — it 'fires' on its own the instant ETH lands.

The attacker hides malicious code HERE. When the victim sends ETH, the attacker's receive() instantly calls withdraw() again.

"Reentrancy" — Breaking Down the Word

RE-ENTRANCY = RE (again) + ENTRANCY (entering)

= *'entering a function AGAIN before the first call has finished.'*

The everyday analogy

- A bank teller hands you cash FIRST...
- ...then turns to update your passbook.
- But before they write it down, you ask to withdraw again.
- The passbook still shows your old (full) balance, so they pay again.
- **Repeat until the drawer is empty.**

The contract version

The vault SENDS ETH (external call) before updating your balance (state).

Sending ETH triggers the attacker's receive() automatically.

receive() calls withdraw() again — re-entering before balance = 0 runs.

Loop drains the whole vault in ONE transaction.

The Root Cause: Wrong Order of Operations

Every withdraw does 3 things. The *ONLY* question is the *ORDER*.

CHECK

Is the request valid? e.g. `require(bal > 0)`

EFFECT

Update internal state. e.g. `balances[user] = 0`

INTERACTION

Talk to the outside world. e.g. send ETH

VULNERABLE order

CHECK → INTERACTION → EFFECT

Sends ETH (interaction) BEFORE zeroing balance (effect). The gap is the bug.

SAFE order (C-E-I)

CHECK → EFFECT → INTERACTION

Balance is zeroed FIRST. When the attacker re-enters, bal is already 0 → require fails → revert.

CEI = Checks-Effects-Interactions — remember this acronym; it's the whole fix.

The Attack, Step by Step (7 Stages)

1

Honest users deposit Alice & Bob deposit 5 ETH each. Vault now holds 10 ETH. Their balances are recorded.

2

Attacker deposits 1 ETH Eve's malicious contract deposits 1 ETH. Vault = 11 ETH. Eve's recorded balance = 1 ETH.

3

Attacker calls withdraw() Eve's contract calls withdraw(). The vault checks: balance > 0? Yes (1 ETH). Proceeds.

4

Vault SENDS 1 ETH first Vault does the external call (sends ETH) BEFORE setting Eve's balance to 0.

5

receive() auto-fires ETH arriving triggers Eve's receive() function automatically — control jumps to attacker code.

6

Re-enter withdraw() receive() calls withdraw() AGAIN. Balance is STILL 1 ETH (not yet zeroed) → check passes → sends again.

7

Loop until empty Steps 4–6 repeat. Each loop drains 1 ETH. Vault hits 0; Eve's contract now holds 11 ETH.

What the 'Recursive Loop' Actually Looks Like

Recursion = a function calling itself. Here the two contracts ping-pong until the money is gone.

```
Vault.withdraw()  
  ↳ sends 1 ETH →  
    Attacker.receive()  
      ↳ calls →  
        Vault.withdraw()  
          ↳ sends 1 ETH ▶  
            Attacker.receive()  
              ↳ calls ▶  
                Vault.withdraw()  
                  ... (until vault = 0)
```

Why it doesn't stop

- The balance is only set to 0 at the END of withdraw().
- But the loop re-enters BEFORE that line is ever reached.
- So the check `require(bal > 0)` keeps passing.
- **It ends only when the vault has no ETH left to send.**

Vulnerable Contract — Line by Line

VulnerableVault.sol

```
mapping(address => uint) public balances;

function withdraw() external {
    uint bal = balances[msg.sender];
    require(bal > 0, "no balance");

    (bool ok, ) =
        msg.sender.call{value: bal}("");
    require(ok, "transfer failed");

    balances[msg.sender] = 0;
}
```

balances mapping

Stores how much ETH each address has put in.

uint bal = ...

Read the caller's current balance into a temp variable.

require(bal > 0)

CHECK: refuse if the caller has nothing.

.call{value: bal}

INTERACTION: send 'bal' ETH to the caller. If caller is a contract, its receive() runs NOW.

balances[...] = 0

EFFECT: zero the balance — but this runs TOO LATE, after the ETH already left.

Attacker Contract — Line by Line

Attacker.sol

```
IVault public vault;
uint public stake;

function attack() external payable {
    stake = msg.value;
    vault.deposit{value: msg.value}();
    vault.withdraw();
}

receive() external payable {
    if (address(vault).balance >= stake) {
        vault.withdraw();
    }
}
```

attack() payable

Entry point. 'payable' lets it receive the 1 ETH we attack with.

vault.deposit(...)

Put our 1 ETH in so we have a non-zero balance (passes the require check).

vault.withdraw()

Start the withdrawal — this is the first domino.

receive()

Auto-runs when the vault sends us ETH. THIS is the trap.

if balance >= stake

Keep going while the vault still has money, then call withdraw() again → the loop.

Fixed Contract — Two Defenses Together

SafeVault.sol

```
import "@openzeppelin/contracts/
  utils/ReentrancyGuard.sol";

contract SafeVault is ReentrancyGuard {

  function withdraw()
    external nonReentrant {
    uint bal = balances[msg.sender];
    require(bal > 0, "no balance");

    balances[msg.sender] = 0;    // EFFECT

    (bool ok, ) =
      msg.sender.call{value: bal}("");
    require(ok, "transfer failed");
  }
}
```

import OpenZeppelin

OpenZeppelin = a library of audited, trusted contract building blocks.

is ReentrancyGuard

Inherit a ready-made lock from that library.

nonReentrant

A 'modifier' that LOCKS the function while it runs. A second entry reverts instantly. (a mutex / lock)

balance = 0 FIRST

CEI fix: zero the balance BEFORE sending ETH. Now re-entry sees 0 and fails the check.

send ETH LAST

Interaction happens only after state is safe. Two layers = defense in depth.

Three Ways to Prevent Reentrancy

1

Checks-Effects-Interactions (CEI)

Always update your state BEFORE any external call. Free, built-in, the #1 fix. Order: check inputs → change state → talk outside.

2

ReentrancyGuard (mutex / lock)

Add OpenZeppelin's nonReentrant modifier. It sets a 'locked' flag on entry; any re-entry hits the lock and reverts. 'Mutex' = MUTual EXclusion lock.

3

Pull-over-Push pattern

Don't let the contract PUSH ETH out in a loop. Instead let each user PULL their own funds in a separate tx, so one call can't chain into many.

Best practice: use CEI + ReentrancyGuard together, and get the code audited.

Real Incidents — This Really Happens

The DAO Hack

2016

~3.6M ETH (~\$60M then) drained via single-function reentrancy. So serious that Ethereum 'hard-forked' (split the chain) to reverse it — birth of Ethereum vs Ethereum Classic.

Curve Finance

2023

~\$70M lost. A bug in the Vyper compiler reintroduced reentrancy in major DeFi pools. Shows even mature, audited projects stay at risk.

Ongoing

2023+

In H1 2023 alone, reentrancy featured in multiple major hacks. Over \$200M+ stolen via this bug class since 2016.

Live Demo Setup — Tools & Install (macOS/Linux)

Foundry = Ethereum dev toolkit (in Rust)

Compiles, tests, and traces contracts locally on your laptop. Commands: forge, anvil, cast. Fast, no internet chain needed.

Forge = the testing tool inside Foundry

Runs your test files and prints a full call trace so you can SEE the recursive withdraw() loop.

Anvil = a local Ethereum node

A pretend blockchain running only on your machine — instant blocks, free fake ETH, perfect for demos.

Cast = command-line Swiss army knife

Read balances / send calls from the terminal, e.g. show the vault going from 10 ETH to 0.

Install (run once)

```
curl -L https://foundry.paradigm.xyz | bash      # download installer
foundryup                                       # install forge, anvil, cast
forge --version                                # verify it worked
```


Live Demo — Build & Run the Exploit

1 • Create project

```
forge init reentrancy-demo && cd reentrancy-demo
```

2 • Add the guard library

```
forge install OpenZeppelin/openzeppelin-contracts --no-commit
```

3 • Add 4 files

```
src/VulnerableVault.sol • src/SafeVault.sol • src/Attacker.sol • test/Reentrancy.t.sol
```

4 • Run the attack (it drains)

```
forge test -vvvv --match-test testReentrancyDrain
```

5 • Run the fix (it reverts)

```
forge test -vvvv --match-test testSafeVaultBlocksAttack
```

'-vvvv' = maximum verbosity: prints the full call trace so the recursive loop is visible on screen.

What to Say While It Runs

Setup

"Alice and Bob deposit 5 ETH each — the vault holds 10 ETH of honest money."

Attack starts

"Eve attacks with just 1 ETH. Watch the trace."

The loop

"See withdraw() calling itself again and again — that's the reentrancy."

Result

"Vault = 0. Eve's contract now holds 11 ETH — she stole everyone's money."

The fix

"Same attack on the SafeVault... it REVERTS. The guard stopped it. Vault keeps its 10 ETH."

Glossary — Every Abbreviation in One Place

ETH

Ether — Ethereum's currency

weismallest ETH unit (1 ETH = 10^{18} wei)**EVM**

Ethereum Virtual Machine — runs contracts

tx

transaction — a signed instruction

CEI

Checks-Effects-Interactions (safe order)

DAO

Decentralized Autonomous Organization

DeFi

Decentralized Finance

mutex

MUTual EXclusion — a lock

uint

unsigned integer (non-negative whole no.)

ABI

Application Binary Interface (call format)

gas

fee/work meter for each operation

.sol

Solidity source-file extension

Team Member Contributions

MEMBER	ROLL NO	CONTRIBUTION
Rohit Patel	G25AIT2089	Lead · attack design · Foundry demo · slides
Sharvan Vittala	G25AIT2099	Vulnerable Vault.sol contract
Sudeb Ghosh	G25AIT2113	Attacker.sol + fallback exploit
Kosuru Yuvaraj	G25AIT2054	Fixed contract (CEI + guard)
Pujan Chakraborty	G25AIT2076	Forge tests + traces
Amit Singh	G25AIT2007	Research · DAO/Curve case studies
Vishnu Priya	G25AIT2128	README + demo video recording

Note: starting split — finalize in the group meeting.

ONE-LINE TAKEAWAY

Update your state before you talk to the outside world.

Reentrancy is a logic bug, not a math bug. **CEI** (state first) + **ReentrancyGuard** (a lock) closes the door.

Group 6 • CSL6010 Cyber Security • Assignment 1 • Reentrancy Attack (Blockchain)

Thank you — Questions?

